Guía paso a paso — Clase 7: Redes convolucionales (puente al aprendizaje profundo)

Temas: imágenes como matrices · correlación cruzada · kernels y detección de bordes · ReLU · aprender un kernel con descenso por gradiente
Duración: ~2 h 35 min
Material: diapositivas `Clase7_Diapositivas` (PDF y PowerPoint, 21 diapositivas) · repositorio https://github.com/datahuba/curso-redes-neuronales
Requisito: Clases 4, 5 y 6 (descenso por gradiente, regla de la cadena, momentum)

Objetivos de la clase

Al terminar, cada alumno podrá:

1. Explicar por qué para imágenes se usan **convoluciones** en lugar de capas densas.
2. Calcular a mano la correlación cruzada de una imagen pequeña con un kernel.
3. Programar la operación de una capa convolucional.
4. Entrenar los números de un kernel con descenso por gradiente.

La guía tiene cuatro partes:

- **Parte A:** preparar las computadoras antes de clase.
- **Parte B:** teoría desde cero.
- **Parte C:** práctica en la computadora.
- **Parte D:** problemas frecuentes.

Agenda

Tiempo	Tema	Diapositivas
0:00	Presentación, objetivos y repaso (10 min)	1–4
0:10	B1. Imágenes y por qué convolución (15 min)	5–6
0:25	B2. Correlación cruzada (25 min)	7–9
0:50	B3. Detector de bordes y ReLU (10 min)	10–11
1:00	C1. Práctica: notebook `3_1` (30 min)	12
1:30	Descanso (10 min)	13
1:40	B4. Aprender el kernel (25 min)	14–18
2:05	C2. Reto: notebook `3_2` y `practica_07` (20 min)	19

Tiempo	Tema	Diapositivas
2:25	Quiz y cierre (10 min)	20–21

PARTE A — Antes de clase (15 min)

En **Anaconda PowerShell Prompt**:

```
cd ~\projects\curso-redes-neuronales
```

```
git pull
```

```
conda activate nni
```

```
python docs/practica_07.py
```

Verificación:

- el ejemplo del pizarrón da `[[27, 27, 0], [27, 27, 0], [27, 27, 0]]`;
- el kernel aprendido es `[[-1, -2, -1], [0, 0, 0], [1, 2, 1]]`, el filtro de Sobel;
- al final dice `Todas las comprobaciones pasaron`.

Con `--graficar` muestra la foto original, sus bordes verticales, sus bordes horizontales y la curva del error.

Checklist:

- [ ] `python docs/practica_07.py --graficar` funciona.
- [ ] En el pizarrón: una cuadrícula de 5 × 5 con números (la imagen del ejemplo de B2) y otra de 3 × 3 (el kernel).
- [ ] Mira de antemano la estrategia del reto en la guía de `3_2`: es la parte más difícil del curso.

PARTE B — Teoría desde cero

Repaso (5 min) — diapositivas 1–4

- "¿Qué hace una capa de una red?" → H = X·W + B y luego la activación.
- "¿Cómo se entrenan los pesos?" → descenso por gradiente, con la regla de la cadena.

Hoy aplicamos lo mismo a **imágenes**.

B1. Imágenes y por qué convolución (15 min) — diapositivas 5–6

Una imagen es una matriz (diapositiva 5)

- Una imagen en escala de grises es una **matriz de números**: 0 es negro y 255 es blanco.
- Una imagen a color son **3 matrices**: rojo, verde y azul.
- La foto de los notebooks mide **256 × 256**: 65 536 números por canal.

¿Por qué no una capa densa? (diapositiva 6)

Con una capa densa (Clase 5), cada píxel se conecta con cada neurona. Con solo 100 neuronas ya serían 65 536 × 100 = **6.5 millones de pesos**. Además:

- trata cada píxel por separado e **ignora que los vecinos están relacionados**;
- un gato en la esquina y un gato en el centro serían problemas distintos.

La convolución usa un filtro pequeño (por ejemplo 3 × 3 = **9 pesos**) que **recorre toda la imagen**. Los mismos 9 pesos se usan en cada posición:

- buscan el **mismo patrón** (un borde, una textura) en todas partes;
- hay muchísimos menos parámetros.

B2. Correlación cruzada (25 min) — diapositivas 7–9

Cómo funciona (diapositiva 7)

1. Coloca el **kernel** (una matriz pequeña) sobre una zona de la imagen.
2. Multiplica cada número del kernel por el píxel que tiene debajo.
3. **Suma** todo: ese es un píxel de la imagen de salida.
4. Desliza el kernel una posición y repite.

En fórmula: $G[i, j] = \Sigma_u \Sigma_v H[u, v] \cdot I[i + u, j + v]$. Los índices u y v van de $-k$ a k; con un kernel de 3 × 3, k = 1.

> Las librerías llaman "convolución" a esta operación. Estrictamente es una correlación cruzada: la convolución matemática voltea el kernel. Para las redes da igual, porque el kernel se aprende.

Ejemplo en el pizarrón (diapositiva 8)

Imagen 5 × 5: oscuro (0) a la izquierda y claro (9) a la derecha. Cada fila es `0 0 9 9 9`.

Kernel 3 × 3: cada fila es `-1 0 1`. Resta la izquierda y suma la derecha.

Píxel de salida en la posición (1, 1), con la ventana en las columnas 0 a 2:

- cada fila aporta $(-1)\cdot 0 + 0\cdot 0 + 1\cdot 9 = 9$;
- son 3 filas: **27**.

Las tres posiciones de la fila:

Columnas de la ventana	Valores	Resultado
0–2	`0 0 9`	27
1–3	`0 9 9`	27

Columnas de la ventana	Valores	Resultado
2–4	`9 9 9`	0 (zona plana: sin borde)

Salida 3 × 3: `[[27, 27, 0], [27, 27, 0], [27, 27, 0]]`.

El kernel **se enciende donde hay un borde vertical** y vale 0 donde la imagen es plana.

Tamaño de la salida (diapositiva 9)

- Sin relleno (**padding 0**), el kernel no puede centrarse en las orillas. La salida pierde k píxeles de cada lado: n − 2k.
  - imagen 5 × 5, kernel 3 × 3 → salida **3 × 3**;
  - imagen 256 × 256 → salida **254 × 254**.
- Con **padding** (rellenar la orilla con ceros), la salida puede medir lo mismo que la entrada.

B3. Detector de bordes y ReLU (10 min) — diapositivas 10–11

El kernel del notebook `3_1` (diapositiva 10)

Cada fila es `-1/3 0 1/3`. Es nuestro ejemplo dividido entre 3:

- da valores **positivos** donde la imagen pasa de **oscuro a claro** (de izquierda a derecha);
- da valores **negativos** donde pasa de claro a oscuro;
- da **≈ 0** en zonas planas.

Después de la ReLU (diapositiva 11)

La ReLU convierte los negativos en 0. Por eso, en la imagen final solo se ven **algunos** bordes verticales: los de oscuro a claro. Tampoco aparecen bordes horizontales, porque este kernel solo compara izquierda contra derecha.

Esa es la respuesta a la pregunta del notebook `3_1`.

B4. Aprender el kernel (25 min) — diapositivas 14–18

La idea (diapositiva 14)

En una red convolucional **nadie elige** los números del kernel: **se aprenden**, igual que los pesos de la Clase 4. Los 9 (o 25) números del kernel son los **pesos** de la capa convolucional.

El gradiente de un peso del kernel (diapositiva 15)

Llamemos:

- G = correlación(K, I);
- P = ReLU(G), la predicción;
- T, la imagen objetivo;
- ECM = promedio de $(P - T)^2$.

El peso K[u, v] se usa en **todas** las posiciones. Por la regla de la cadena, su gradiente **suma la contribución de cada posición**:

$\partial ECM/\partial K[u, v] = (2/N) \cdot \Sigma\ (P - T)[i, j] \cdot [G[i, j] > 0] \cdot I[i + u, j + v]$

Se lee: el error en cada píxel, por si la ReLU dejó pasar (1 o 0), por el píxel que se multiplicó por ese peso. Luego: **K ← K − η·gradiente**.

Las trampas (diapositiva 16)

1. **ReLU muerta:** si al inicio todos los G son negativos, la ReLU da 0 en todos lados y ningún peso recibe gradiente. Con fotos (todos los píxeles positivos) y un kernel aleatorio pasa muy seguido.
2. **Escala:** con píxeles de 0 a 255 los gradientes son enormes. Hay que normalizar a 0–1.
3. **Lentitud:** calcular píxel por píxel con `for` tarda minutos por época.

Los trucos que funcionan:

- normalizar (dividir entre 255);
- **centrar** la entrada (restarle su promedio);
- **vectorizar**;
- usar **momentum** (Clase 6).

`practica_07.py` muestra el efecto de centrar: con la imagen sin centrar, el entrenamiento **diverge**.

Vectorizar (diapositiva 17)

En lugar de recorrer píxeles, se preparan las versiones **desplazadas** de la imagen: 9 para un kernel de 3 × 3, o 25 para uno de 5 × 5.

- La correlación es la suma de cada versión por su peso.
- El gradiente de cada peso es un promedio sobre una matriz completa.
- Todo con NumPy: milisegundos por época.

Resultados (diapositiva 18)

- **`practica_07.py`:** sobre una imagen de ruido centrada, el descenso **recupera exactamente** el filtro de Sobel `[[-1, -2, -1], [0, 0, 0], [1, 2, 1]]`, con ECM de 0.97 a ≈ 0.
- **Reto** `3_2` (foto real, kernel de 5 × 5, ReLU, momentum β = 0.9, η = 2, 2000 épocas):
  - el ECM baja de ≈ 31 600 a ≈ **29**;
  - el kernel aprendido es un Sobel en el centro (fila −0.86, −1.77, −0.89 arriba y 0.93, 1.79, 0.84 abajo);
  - la imagen predicha es casi idéntica al objetivo.

Mensaje final: una red convolucional profunda hace esto mismo con **muchos** kernels y **muchas** capas. Las primeras capas aprenden bordes; las siguientes, formas; las últimas, objetos.

PARTE C — Práctica en la computadora

Guías celda por celda: `docs/guias_notebooks/` (NB_3_1 y NB_3_2).

C1. `notebooks/3_1_capa_convolucional.ipynb` (30 min, 4 puntos) — diapositiva 12

- **`correlacionPixel`:** dos `for` en `range(-k, k + 1)`, acumulando `H[u+k, v+k] * I[i+u, j+v]`. `resultado` debe ser un número de NumPy, por ejemplo `np.array(sumatoria)`.

- `G`: ceros del mismo tamaño que I; al final, recórtalo a `G[k:m-k, k:n-k]`.
- **Pregunta:** ver B3.

✅ **Comprobación:**

- la salida mide **254 × 254**;
- tras la ReLU se ven solo algunos bordes verticales, como en `files/ccnout.png`.

C2. Reto `notebooks/3_2_entrenar_capa_conv.ipynb` y `practica_07.py` (20 min) — diapositiva 19

- Sigue la estrategia de B4. El detalle está en la guía NB_3_2.
- Si el tiempo no alcanza, deja el reto de tarea y muestra en vivo:

```
python docs/practica_07.py --graficar
```

Quiz y cierre (10 min) — diapositivas 20–21

Pregunta	Respuesta
¿Por qué no usar capas densas para imágenes?	Demasiados pesos, e ignoran que los píxeles vecinos se relacionan
¿Cuánto mide la salida de una imagen 7 × 7 con un kernel 3 × 3, sin padding?	5 × 5
¿Qué detecta el kernel con filas −1 0 1?	Bordes verticales (de oscuro a claro, hacia la derecha)
¿Por qué se ven solo algunos bordes tras la ReLU?	La ReLU elimina los valores negativos (bordes de claro a oscuro)
¿Qué son los "pesos" de una capa convolucional?	Los números del kernel

Para llevar:

1. Convolución = un filtro pequeño que recorre la imagen con los mismos pesos.
2. Los kernels detectan patrones (bordes, texturas), y **se aprenden** con descenso por gradiente.
3. Las redes profundas apilan muchas capas convolucionales: es el siguiente curso.

Tarea:

1. Terminar `3_1` (4 puntos).
2. Intentar el reto `3_2`, opcional y con puntos extra.
3. En `practica_07.py`, cambia el kernel objetivo por el detector vertical `[[-1, 0, 1]] * 3` y comprueba que también lo aprende.

Rúbrica:

Notebook	Puntos
3_1	4

Notebook	Puntos
3_2 (reto)	Extra

PARTE D — Problemas frecuentes

Síntoma	Causa	Solución
`AttributeError: 'float' object has no attribute 'astype'`	`resultado` es un número normal de Python	`np.array(sumatoria)`
La imagen sale con un marco negro	No recortaste G	`G[k:m-k, k:n-k]`
La predicción del reto es toda negra	ReLU muerta	Centra la entrada; ver B4
El ECM explota o sale `nan`	Tasa grande o datos sin normalizar	Divide entre 255 y baja η
Cada época tarda minutos	Cálculo píxel por píxel	Vectoriza con versiones desplazadas
`FileNotFoundError: files/Lenna.png`	Notebook fuera de la carpeta del repo	Abre la carpeta `curso-redes-neuronales`